ボイヤ・ムーア法

文字列の検索アルゴリズム（長い文章の中から、指定したキーワードを見つける仕組み）の中でも、圧倒的なスピードを誇るのが「ボイヤ・ムーア法（BM法）」です。

多くの検索アルゴリズムが「文章の先頭」から順番に調べていくのに対し、ボイヤ・ムーア法は「キーワードの後ろ（末尾）から逆向きに調べる」という、一見へそ曲がりなアプローチをとります。しかし、この「後ろから調べる」ことこそが、爆発的なスピードを生み出す秘密なのです。

I. ロジック解説

1. 最大の特徴：文字を「後ろから」比較する

例えば、大きな文章の中から「**APPLE**」という5文字のキーワードを探したいとします。

普通のやり方（力まかせ法）は、1文字目の「A」が合っているか調べますが、ボイヤ・ムーア法は、**いきなりキーワードの最後の文字「E」から調べ始めます。**

```
文章：    [ ? ] [ ? ] [ ? ] [ ? ] [ X ] [ ? ] [ ? ] ...
                      └──────── キーワードの末尾を合わせる
キーワード：A     P     P     L     E
                                  ▲（ここから調べる！）
```

---

2. なぜ後ろから調べると速いの？（仕組みの鍵）

「後ろから調べて、もし違っていたとき」を想像してみてください。ここにボイヤ・ムーア法が最強と呼ばれる理由があります。

例えば、調べた位置の文章の文字が、キーワードにまったく含まれていない「X」だったとします。

💡 普通のやり方なら……

「1文字目が違ったから、全体を1文字だけ右にずらして、また最初から調べよう」と、地道に1文字ずつ進みます。

🚀 ボイヤ・ムーア法なら……

「最後の文字を調べたら『X』だった。**『X』は『APPLE』の中に1文字も含まれていない** よね。ということは、今重ねている範囲（5文字分）のどこにキーワードをずらしても、絶対に『X』と重なって不合格になるはずだ！」

とコンピュータが一瞬で判断します。その結果、1文字ずつずらすのではなく、一気に5文字分ドカンと右にジャンプ（スキップ）させます。

```
【1回目の比較：不合格！】
文章：      ？  ？  ？  ？  X  ？  ？  ？  ？  ？
キーワード：A   P   P   L   E
                        ▲（違った！しかも X は APPLE に無い！）
```

```
【一気に5文字ジャンプ！】
文章：      ？  ？  ？  ？  X  ？  ？  ？  ？  E
                           └──────── 新しい重ね合わせ位置
キーワード：                A   P   P   L   E
                                    ▲（次のお出番）
```

このように、**「どうせ一致しない場所」をまとめてゴミ箱に放り込み、大きな歩幅でジャンプしていく** ため、文章が長ければ長いほど、他のアルゴリズムを置き去りにするほどの超高速化が実現します。

---

3. もし途中の文字が含まれていたら？（ずらし表の作成）

「キーワードの中に含まれている文字」が途中で見つかった場合は、その文字がうまく重なる位置まで、少しだけ慎重にジャンプします。

例えば、「**APPLE**」を探していて、末尾を調べたら「**P**」だった場合：「APPLE」の中には「P」が2箇所（2文字目と3文字目）にありますよね。この場合は、**一番後ろにある「P」がぴったり重なる位置まで、2文字分だけ右にずらし** ます。

```
文章：      ？  ？  ？  ？  P  ？  ？
キーワード：A   P   P   L   E
                    ▲（違った！でも P はキーワードの中にある！）
```

【Pが重なる位置までずらす】

```
文章：      ？  ？  ？  ？  P  ？  ？
キーワード：        A   P   P   L   E
                    ▲（重なった！ここから再開）
```

プログラムを動かす前には、あらかじめ「どの文字に出会ったら、何文字スキップするか」をまとめた「ずらし表（スキップテーブル）」を内部で自動作成しておきます。

APPLE のずらし表

- **文字列の長さ - 先頭からの位置** 分ずらす。Aなら、4(= 文字列の長さ 5 - 先頭からの位置 1)ずらす。

```
文章：       ?   ?   ?   ?   A   ?   ?
キーワード： A   P   P   L   E
ずらした後：                 A   P   P   L   E
ずらし量：   0   1   2   3   4
```

文字	移動文字数	コメント
A	4	
P	2	同じ文字が二つあれば後ろを取る
L	1	
E	5	末尾は0でなく文字列の長さ
その他	5	その他も文字列の長さ

- 末尾の扱い： ずらし量が 0 だと検索位置がまったく進まず（まったくずれないので）、無限ループに陥ってしまう。 そのため、パターン自体の長さ（初期値）である 5 がそのまま割り当てられる。

FREEZER のずらし表

特殊な例: 同じ文字が複数あっても、それが文字列の末尾なら、そのひとつ前を取る

文字	移動文字数	コメント
F	6	
R	5	同じ文字が二つ以上でも、末尾は取らない
E	1	同じ文字が二つ以上なら一番後ろを取る
Z	2	
その他	7	

まとめ：イメージ

以下のような日常の例えでまとめてみると、直感的に納得できるかもしれません。

> 「パラパラ漫画や本のページから、特定の単語を探すときをイメージしてみて。 1文字ずつ指でなぞるように読む（普通の検索）のは時間がかかるよね。 ボイヤ・ムーア法は、**『あ、このページには探したい単語の文字が1つも入ってないや！』と気づいた瞬間、その数ページをごそっと束ねてガバッとめくる（ジャンプする）** ような読み方なんだ。 だから、**『探したいキーワードが長ければ長いほど』、そして『一致しない文字が多ければ多いほど』、大股でジャンプできるから速くなる** という、とっても賢くてエコなアルゴリズムなんだ」

基本情報の試験でも「後ろから比較する」「一致しないときに大きくスキップする」という特徴が非常によく出題されます。

II. 実装例

ボイヤ・ムーア法（BM法）のアルゴリズムを、基本情報技術者試験で使われる **疑似言語形式** と、実際の開発で使われる **C言語形式** の2パターンで実装しました。

どちらの実装も **「①ずらし表（スキップテーブル）を作る処理」** と **「②それを使って実際に検索する処理」** の2つのパーツに分かれていることを意識して、コードを読み解いてください。

---

1. 疑似言語による実装

現在の試験（科目B）の仕様に合わせ、クラスやドット表記、`if-else` 等の標準的な記法で記述しています。文字コード（文字の種類）は英数字を想定し、256個の要素を持つ配列としてずらし表を表現します。

```
/* ボイヤ・ムーア法で文字列を検索する関数 */
/* 返り値：見つかった場合はその先頭インデックス、見つからない場合は -1 */
○ 整数型: bmMatch(文字列型: text, 文字列型: pattern)
    整数型: textLen ← textの長さ
    整数型: patLen ← patternの長さ
    整数型: skip[256]
    整数型: i, j, textCharNum

    /* --- ① ずらし表（スキップテーブル）の作成 --- */
    /* すべての文字のスキップ数を、一律「パターンの長さ」で初期化 */
    for (i を 0 から 255 まで 1 ずつ増やす)
        skip[i] ← patLen
    endfor

    /* パターンに含まれる文字のスキップ数を上書き計算 */
    for (i を 0 から patLen - 2 まで 1 ずつ増やす)
        /* 文字を配列の添字（0〜255）に変換 */
        textCharNum ← patternの i 番目の文字の文字コード
        skip[textCharNum] ← patLen - 1 - i
    endfor

    /* --- ② 検索処理 --- */
    /* i はテキスト側の注目位置（パターンの末尾と重なる位置からスタート） */
    i ← patLen - 1

    while (i ＜ textLen)
        /* パターンの後ろ（末尾）から前に向かって比較 */
        j ← patLen - 1

        /* 文字が一致している間、前方に遡って比較を続ける */
        while (j ≧ 0 and textの i 番目の文字 ＝ patternの j 番目の文字)
            i ← i - 1
            j ← j - 1
        endwhile

        /* パターンの先頭（j < 0）まで全部一致したら検索成功！ */
        if (j ＜ 0)
            return i + 1  /* 見つかった先頭の位置を返す */
```

```
        endif

        /* 不一致だった場合、ずらし表を見て一気にジャンプする */
        /* ただし、比較が進んだ分（patLen - 1 - j）を考慮して次のスタート位置を決める
*/
        textCharNum ← textの i 番目の文字の文字コード
        if (skip[textCharNum] > patLen - 1 - j)
            i ← i + skip[textCharNum]
        else
            i ← i + (patLen - 1 - j) + 1
        endif
    endwhile

    return -1  /* 最後まで見つからなかった場合 */
```

---

2. C言語による実装

上記のアプローチを、そのまま実用的なC言語のコードに落とし込んだものです。安全に1文字（`unsigned char`）を配列の添字として扱えるようにしています。

```c
#include <stdio.h>
#include <string.h>

#define ALPHABET_SIZE 256

/* マクロで2つの値の大きい方を選択（不一致時のポインタ逆流防止用） */
#define MAX(a, b) ((a) > (b) ? (a) : (b))

int bmMatch(const char *text, const char *pattern) {
    int textLen = strlen(text);
    int patLen = strlen(pattern);
    int skip[ALPHABET_SIZE];

    /* パターンが空、またはテキストより長い場合はエラー */
    if (patLen == 0 || patLen > textLen) return -1;

    /* --- ① ずらし表（スキップテーブル）の作成 --- */
    for (int i = 0; i < ALPHABET_SIZE; i++) {
        skip[i] = patLen;
    }
    for (int i = 0; i < patLen - 1; i++) {
        skip[(unsigned char)pattern[i]] = patLen - 1 - i;
    }

    /* --- ② 検索処理 --- */
    int i = patLen - 1; /* テキスト側の注目位置（初期値はパターンの末尾位置） */

    while (i < textLen) {
        int j = patLen - 1; /* パターン側の注目位置（末尾からスタート） */
```

```
        /* 後ろから前に向かって一致チェック */
        while (j >= 0 && text[i] == pattern[j]) {
            i--;
            j--;
        }

        /* 完全に一致した場合の処理 */
        if (j < 0) {
            return i + 1; /* 一致した先頭のインデックスを返却 */
        }

        /* 不一致の場合、ずらし表に従って大股ジャンプ */
        /* すでに後ろ向きに何文字か一致して引き返した分（patLen - 1 - j）の移動量を相
殺する */
        i += MAX(skip[(unsigned char)text[i]], patLen - j);
    }

    return -1; /* 見つからなかった場合 */
}

int main(void) {
    const char *text = "APPLES AND BANANAS ORANGE";
    const char *pattern = "BANANA";

    int result = bmMatch(text, pattern);

    if (result != -1) {
        printf("見つかりました！ インデックス: %d\n", result);
        printf("該当箇所: %s\n", &text[result]);
    } else {
        printf("見つかりませんでした。\n");
    }

    return 0;
}
```

💡 理解のツボ：最後のジャンプの計算式

不一致時のジャンプ部分にある `MAX`（または疑似言語の `if` 分岐）の処理は、一見複雑で混乱しやすいポイントです。ここはこう考えてください。

> 「後ろからせっかく2文字一致して、3文字目で不一致になったとするよね。 このとき、ポインタ `i` はすでに2文字分**左（前）に引き返してしまっている**状態なんだ。 だから、ずらし表の数字をそのまま足すだけだと、引き返した分だけジャンプの歩幅が小さくなってしまう（最悪の場合、ポインタが後ろに逆戻りしてしまう）。 それを防ぐために、**『ずらし表の指示』と『引き返した歩数 ＋ 1マス』を比べて、必ずポインタが右（後ろ）に進むように安全弁をつけているんだよ**」

この一言を添えてあげると、ただコードを暗記するのではなく、アルゴリズムの堅牢性を深く理解できるようになります。

注意

「このコードは日本語でも動くのか？」という疑問を持った方もいるかもしれません。

上の実装例は ASCIIコードを前提としているので、日本語には対応していません。

英語（ASCIIコード）は、1つのアルファベットがちょうど1つの箱（1バイト）に収まるので、先ほどのプログラムで完璧に動きます。

しかし、日本の文字は複雑だから、1文字を表現するのに3つも4つも箱を合体させて使っています。つまり「1バイトの文字もあれば4バイトの文字もある」という仕組みです。

だから、箱1個(1バイト)単位で大雑把にジャンプする仕組みのままだと、文字の途中で噛み合っちゃってバグが起きます。

もし日本語でやりたいときは、ワイド文字(たとえば32バイト固定のUTF-32)などを使って箱を大きくして(一箱32バイト)、『1文字＝箱1個』になるように仕組みに変身させてから、同じアルゴリズムを動かすのがプロの技です。